

Extend `std::is_within_lifetime`

Document #: P3450R0 [\[Latest\]](#) [\[Status\]](#)
Date: 2024-10-14
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
[<barry.revzin@gmail.com>](mailto:barry.revzin@gmail.com)

Contents

[1 Introduction](#)

[2 Proposal](#)

[3 Wording](#)

[3.1 Feature-test Macro](#)

[4 References](#)

§ 1 Introduction

In [\[P2641R4\]](#), I proposed adding the metafunction

```
template<class T>
constexpr bool is_within_lifetime(const T* p) noexcept;
```

To check if `p` points to an object that is usable in constant expressions. This is a narrowly useful function, but it is useful.

I recently ran into cases where a very slight extension of it could make it more useful. In attempting to implement `constexpr std::format` [\[P3391R0\]](#), both within `{fmt}` and `libstdc++`, I ran into the same issue. There is a situation where, during `parse()`, at runtime an object of type `Base` is constructed and used except that, during constant evaluation, an object of type `Derived` is used instead. In both libraries, the code assumes that if we're being constant evaluated, we have a `Derived` and can thus safely downcast. However, in attempting to make the `format()` function `constexpr` in addition to `parse()`, I had to deal with the fact that the object in question now has to be a `Base` — not a `Derived`. That makes this downcast ill-formed.

This only happens at compile-time though. The compiler knows whether there is a `Derived` or `Base` there. I simply needed a way to check to see if I could cast down to that `Derived` before doing so. But there is no such way to check this. The function `is_within_lifetime` seems like it would solve this problem for me, if I could only call it as `is_within_lifetime(static_cast<Derived*>(p))`. Unfortunately, it is the `static_cast` itself that is disallowed — so the result of such a check would be either `true` or ill-formed. Not a very useful check!

A very slight change to the specification of this function would let it solve this problem too.

§ 2 Proposal

Extend `std::is_within_lifetime` so that it takes an additional, defaulted template parameter, allowing you to check not just that `p` points to a valid object but also to check for a derived type. Instead of:

```
template<class T>
constexpr bool is_within_lifetime(const T* p) noexcept;
```

It'll become:

```
template<class U=void, class T>
constexpr bool is_within_lifetime(const T* p) noexcept;
```

And check if `p` is a pointer to an object within its lifetime that can be converted to `const U*` as a constant expression.

§ 3 Wording

Add to 21.3.3 [\[meta.type.synop\]](#):

```
// all freestanding
namespace std {
    // ...

    // [meta.const.eval], constant evaluation context
    constexpr bool is_constant_evaluated() noexcept;
- template<class T>
+ template<class U=void, class T>
    constexpr bool is_within_lifetime(const T*) noexcept;
}
```

And change the specification of this function:

```
- template<class T>
+ template<class U=void, class T>
    constexpr bool is_within_lifetime(const T* p) noexcept;
```

- * *Mandates:* `static_cast<const U*>(p)` is well-formed.
- 3 *Returns:* `true` if `p` is a pointer to an object that is within its lifetime ([basic.life]) and `static_cast<const U*>(p)` is a core constant expression; otherwise, `false`.
- 4 *Remarks:* During the evaluation of an expression `E` as a core constant expression, a call to this function is ill-formed unless `p` points to an object that is usable in constant expressions or whose complete object's lifetime began within `E`.

§ 3.1 Feature-test Macro

Bump the feature-test macro in 17.3.2 [[version.syn](#)]:

```
- #define __cpp_lib_within_lifetime 202306L // also in <type_traits>  
+ #define __cpp_lib_within_lifetime 2024XXL // also in <type_traits>
```

§ 4 References

[P2641R4] Barry Revzin. 2023-06-15. Checking if a union alternative is active.

<https://wg21.link/p2641r4>

[P3391R0] Barry Revzin. 2024-09-12. constexpr std::format.

<https://wg21.link/p3391r0>